

Practice Question Set

Core Definitions & Operators

1. What is a pointer variable?
2. Explain the purpose of the address-of operator (&) and list its other technical names.
3. Explain the purpose of the indirection operator (*) and list its alternative technical names.
4. Can a programmer manually change or select the specific memory address allocated to a standard variable?
5. Write the correct syntax to declare a pointer variable.

Pointer Types & Advanced Concepts

6. What is a pointer-to-pointer (or double pointer), and why is it used?
7. What is a Void Pointer, and what unique characteristic makes it different from standard pointers?
8. What is a Null Pointer, and what happens if you attempt to dereference it?
9. What is a Wild Pointer, and how does it affect a program's stability?
10. What is a Dangling Pointer, and what step can a programmer take to fix it?

Important Interview/Viva Questions

11. Why does the assignment operation `&x = 7;` generate a compilation error in C?
12. Explain the fundamental architectural difference between Call by Value and Call by Reference.
13. How does the compiler map multi-dimensional array indices to pointer notations? Convert `a[3][5][2]` completely into pointer arithmetic notation.
14. Which specific mathematical operations are strictly prohibited or invalid when working with two pointer variables?
15. Explain how adding an integer to a pointer (e.g., `p = p + 3`) scales behind the scenes.
16. What occurs when you subtract one pointer variable from another pointer variable?
17. Why is it impossible to directly dereference a void pointer (`*vp`) without type-casting?
18. Compare the behavioral difference between two uninitialized local pointers versus two local pointers explicitly set to NULL.
19. Is the initialization `int *ptr[] = {11, 22, 33, 44};` valid in C? Explain the correct way to link an array of pointers to data.
20. Why is defining `void main()` discouraged in professional C programming? What should be used instead?

Answers

1. What is a pointer variable?

Answer: A pointer is a specialized variable that stores the memory address of another variable rather than holding an ordinary data value.

2. Explain the purpose of the address-of operator (&) and list its other technical names.

Answer: The & operator is a unary operator used to retrieve the exact physical memory location (address) of a variable. It is also known as the **referencing operator** or **address-of operator**.

3. Explain the purpose of the indirection operator (*) and list its alternative technical names.

Answer: The * operator is a unary operator prefixed to a pointer variable to access or modify the data value stored at the memory location held by that pointer. Its alternative technical terms are the **indirection operator** or **dereferencing operator**.

4. Can a programmer manually change or select the specific memory address allocated to a standard variable?

Answer: No. Programmers cannot directly change, manipulate, or decide the memory addresses assigned to variables; addresses are designated strictly by the system during program execution.

5. Write the correct syntax to declare a pointer variable.

Answer:

Syntax: data_type *pointer_name;

This statement explicitly informs the compiler that the declared variable identifier is intended to function as a pointer pointing to the specified data type block.

6. What is a pointer-to-pointer (or double pointer), and why is it used?

Answer: A pointer-to-pointer (declared using) is a variable that stores the memory address of another pointer variable, rather than pointing to an ordinary data variable.

7. What is a Void Pointer, and what unique characteristic makes it different from standard pointers?

Answer: A void pointer (declared as void *vp;) is a generic pointer that does not have any specific associative data type. Unlike standard pointers—which can only store addresses of variables matching their declared type—a void pointer can store the memory address of any variable of any data type (e.g., int, float, or char).

8. What is a Null Pointer, and what happens if you attempt to dereference it?

Answer: A null pointer is a pointer variable explicitly assigned to NULL or 0, indicating that it does not point to any valid object or memory location. If you attempt to dereference a null pointer (*ptr), it will instantly **crash the program** because it refers to the reserved 0-th memory layer, which is strictly off-limits for user manipulation.

9. What is a Wild Pointer, and how does it affect a program's stability?

Answer: An uninitialized pointer variable acting alone in a block is called a wild pointer. It contains an unpredictable garbage address, and dereferencing it creates a highly unstable scenario: it can print garbage values, crash the application randomly, or introduce hidden bugs into your software.

10. What is a Dangling Pointer, and what step can a programmer take to fix it?

Answer: A dangling pointer is a variable that continues to store the memory address of an in-active, freed, or deallocated area of system memory. To clean up after deallocation and avoid severe undefined bugs, a programmer must reset the pointer variable explicitly to NULL.

Important Interview Answers

11. Why does the assignment operation `&x = 7;` generate a compilation error in C?

Answer: The expression `&x` evaluates directly to a fixed numerical memory location, meaning it is a **constant value**, not a variable. The structural rules of C require a modifiable variable to stand on the left-hand side of an assignment operator; attempting to assign a new value to a constant expression triggers a mandatory **compilation error**.

12. Explain the fundamental architectural difference between Call by Value and Call by Reference.

Answer:

Call by Value: Passes an isolated duplicate copy of the data value to the called function, meaning modifications inside the function body have absolutely zero impact on the original variables.

Call by Reference: Passes the direct memory address of the original variable, enabling the function to modify the actual data location directly via pointer dereferencing.

13. How does the compiler map multi-dimensional array indices to pointer notations? Convert `a[3][5][2]` completely into pointer arithmetic notation.

Answer: The index notation `[i]` converts to `*(base + i)`. For multi-dimensional grids, this conversion layer unpacks sequentially from left to right:

`a[3]` maps directly to `*(a + 3)`

`a[3][5]` opens up into `*(*(a + 3) + 5)`

`a[3][5][2]` fully expands out to: `*(**(*a + 3) + 5) + 2)`

14. Which specific mathematical operations are strictly prohibited or invalid when working with two pointer variables?

Answer: You cannot add two pointer variables together ($p + q$), multiply pointers ($p * q$), or perform division on pointers (p / q). Attempting to compute operations with two address boundaries will trigger a compilation error.

15. Explain how adding an integer to a pointer (e.g., $p = p + 3$) scales behind the scenes.

Answer: Adding an integer n to a pointer does not increment the address by n bytes; instead, it shifts the pointer forward by n structural element positions. The mechanical address scaling formula is executed as:

New Address = Current Address + ($n * \text{sizeof}(\text{data_type})$)

For an integer pointer at address 1000, $p + 3$ results in $1000 + (3 * 4) = 1012$ bytes.

16. What occurs when you subtract one pointer variable from another pointer variable?

Answer: Subtracting one pointer from another evaluates to the **number of element positions standing between the two addresses**, provided both pointers share the same base data type. The underlying compiler evaluates this using the relation:

$$\text{Distance} = \frac{\text{Address 2} - \text{Address 1}}{\text{sizeof}(\text{data_type})}$$

For example, if $q = 1024$ and $p = 1008$, the physical distance is 16 bytes, which divides by $\text{sizeof}(\text{int})$ (4 bytes) to return a final answer of 4.

17. Why is it impossible to directly dereference a void pointer (*vp) without type-casting?

Answer: A void pointer lacks any associative data type identifier. Because of this, the compiler cannot determine the size of the underlying memory block or interpret how to read the data bytes stored there. To safely access the data, you must explicitly type-cast it first (e.g., $*(\text{int} *)vp$).

18. Compare the behavioral difference between two uninitialized local pointers versus two local pointers explicitly set to NULL.

Answer:

Uninitialized Pointers: If you declare $\text{int} *ptr1$; $\text{int} *ptr2$;, they contain random garbage addresses. There is absolutely no guarantee that $ptr1 == ptr2$ or that their contents behave predictably.

NULL Pointers: If initialized as `int *ptr1 = NULL; int *ptr2 = NULL;`, both point to the safe 0th layer. This guarantees that the evaluation `ptr1 == ptr2` will always hold true, providing a clean condition check for memory safety.

19. Is the initialization `int *ptr[] = {11, 22, 33, 44};` valid in C? Explain the correct way to link an array of pointers to data.

Answer: No, initializing an array of pointers with raw integer constants is invalid and throws compiler warnings. An array of pointers must explicitly store memory address values. The correct, compliant approach is to store variables in a standard data array and assign their addresses to the pointer array:

```
int a[] = {11, 22, 33, 44}; int *ptr[] = {a, a + 1, a + 2, a + 3}; ----> Stores the actual memory addresses
```

20. Why is defining `void main()` discouraged in professional C programming? What should be used instead?

Answer: Defining `void main()` is discouraged because it does not provide an exit status to the operating system. The proper, standard-compliant convention is to define **`int main()`** and include a **`return 0;`** statement at the end of the function body to signal to the environment that the program executed and terminated successfully.